# TRABALHO PRÁTICO
# META 2

DEIS – Programação Orientada a Objetos – 2025/2026


Lecionado por:

João António Pereira Almeida Durães– jduraes@isec.pt

Realizado por:

Pedro Saraiva – a2023146226@isec.pt
Rúben Costa – a2024132692@isec.pt

# Índice

1. Introdução e Enquadramento do Problema ..... 2
2. Descrição Geral do Sistema Desenvolvido ..... 2
2.1. Objetivos do Trabalho ..... 2
2.2. Requisitos Funcionais e Restrições ..... 3
3. Modelação Orientada a Objetos ..... 3
3.1 Estrutura Geral do Projeto ..... 4
3.2 Hierarquia de Classes e Relações ..... 5
4. Descrição das Principais Classes ..... 6
4.1 Classe Simulador ..... 6
4.2 Classe Jardim ..... 6
4.3 Classe Jardineiro ..... 6
4.4 Classe Planta e Especializações ..... 7
4.5 Classe Ferramenta e Especializações ..... 7
4.6 Características das Plantas e Ferramentas ..... 8
4.7 Classes de Apoio ..... 9
5. Funcionamento da Simulação ..... 9
6. Conclusão ..... 10

# 1. Introdução e Enquadramento do Problema

A programação orientada a objetos (POO) é um paradigma amplamente utilizado no desenvolvimento de software, baseado na modelação de entidades através de objetos, recorrendo a encapsulamento, herança e polimorfismo.

No âmbito da unidade curricular de Programação Orientada a Objetos, foi desenvolvido um sistema de simulação de um jardim, composto por diferentes tipos de plantas, ferramentas e um jardineiro responsável pela sua gestão. Este problema permite aplicar de forma prática os principais conceitos da POO.

O sistema simula a evolução do estado do jardim ao longo do tempo, em função das ações executadas e das regras associadas a cada objeto. O objetivo do trabalho consiste na implementação de uma solução orientada a objetos bem estruturada, clara e coerente com as boas práticas abordadas na disciplina.

# 2. Descrição Geral do Sistema Desenvolvido

## 2.1. Objetivos do Trabalho

O principal objetivo deste trabalho é desenvolver um sistema de simulação utilizando o paradigma da programação orientada a objetos, aplicando de forma prática os conceitos lecionados na unidade curricular.

Em particular, pretende-se:

1. Modelar um conjunto de entidades relacionadas com a gestão de um jardim, recorrendo a classes bem definidas;
2. Aplicar corretamente os princípios de encapsulamento, herança e polimorfismo;
3. Implementar interações entre objetos de forma coerente e controlada;
4. Garantir uma estrutura de código clara, modular e de fácil manutenção;
5. Simular a evolução do sistema ao longo do tempo, refletindo alterações no estado dos objetos.

## 2.2. Requisitos Funcionais e Restrições

O sistema desenvolvido deve permitir a simulação de um jardim, assegurando a criação, gestão e interação entre as diferentes entidades envolvidas. Em particular, devem ser suportadas as seguintes funcionalidades:

- Representação de um jardim constituído por posições e plantas;
- Existência de diferentes tipos de plantas, com comportamentos específicos;
- Utilização de ferramentas que permitem interagir com as plantas;
- Presença de um jardineiro responsável pela execução das ações disponíveis;
- Evolução do estado do jardim ao longo do tempo, de acordo com regras definidas.

Relativamente às restrições, o desenvolvimento do sistema deve:

- Respeitar o paradigma da programação orientada a objetos;
- Utilizar a linguagem C++;
- Evitar dependências desnecessárias e manter uma estrutura de projeto simples;
- Cumprir os requisitos e limitações definidos no enunciado do trabalho prático.

# 3. Modelação Orientada a Objetos

A modelação do sistema foi realizada segundo os princípios da programação orientada a objetos, identificando as principais entidades do problema e representando-as através de classes com responsabilidades bem definidas. Esta abordagem permitiu estruturar o sistema de forma modular e extensível.

O domínio do problema foi dividido em componentes essenciais, nomeadamente o jardim, as plantas, as ferramentas e o jardineiro. Cada uma destas entidades corresponde a uma abstração do mundo real e possui comportamento próprio, bem como relações claras com os restantes elementos do sistema.

Foram utilizados mecanismos de herança para representar especializações de plantas e ferramentas, permitindo reutilização de código e diferenciação de comportamentos. O encapsulamento foi garantido através do controlo do acesso aos atributos internos, e o polimorfismo permitiu tratar objetos de diferentes subclasses de forma uniforme.

Esta modelação assegura uma separação clara de responsabilidades, facilitando a compreensão do código, a sua manutenção e a evolução futura do sistema.

## 3.1 Estrutura Geral do Projeto

A organização do projeto segue uma estrutura modular, baseada na separação entre ficheiros de cabeçalho e implementação, refletindo diretamente a modelação orientada a objetos adotada. Cada classe principal do sistema possui um par de ficheiros (.h / .cpp), garantindo uma clara distinção entre interface e lógica de funcionamento.

```
TP_POO/
├── main.cpp
├── Simulador.h / Simulador.cpp
├── Jardim.h / Jardim.cpp
├── Posicao.h / Posicao.cpp
├── Jardineiro.h / Jardineiro.cpp
├── Settings.h / Settings.cpp
├── Planta.h / Planta.cpp
│   ├── Cacto.h / Cacto.cpp
│   ├── Roseira.h / Roseira.cpp
│   ├── Ervadaninha.h / Ervadaninha.cpp
│   └── PlantaExotica.h / PlantaExotica.cpp
└── Ferramenta.h / Ferramenta.cpp
    ├── Regador.h / Regador.cpp
    ├── Adubo.h / Adubo.cpp
    ├── Tesoura.h / Tesoura.cpp
    └── FerramentaZ.h / FerramentaZ.cpp
```

## 3.2 Hierarquia de Classes e Relações

A hierarquia de classes do sistema foi definida de forma a representar as entidades principais do domínio e as respetivas relações, recorrendo à herança para modelar especializações e à agregação para estruturar as interações entre objetos.

### Hierarquia de Plantas:

Planta

├── Cacto

├── Roseira

├── Ervadaninha

└── PlantaExotica

### Hierarquia de Ferramentas

Ferramenta

├── Regador

├── Adubo

├── Tesoura

└── FerramentaZ

### Relações entre Classes Principais

Jardineiro ── utiliza ──> Ferramenta

└── interage com ──> Jardim ── contém ──> Planta

└── contém ──> Posicao

# 4. Descrição das Principais Classes

## 4.1 Classe Simulador

A classe Simulador é responsável pelo controlo global da aplicação. Compete-lhe inicializar o jardim, o jardineiro e os restantes componentes, bem como gerir o ciclo de execução da simulação. Esta classe funciona como o ponto central de coordenação, garantindo a correta evolução do sistema ao longo do tempo.

Simulador

├── controla ciclo da simulação

├── inicializa objetos principais

└── coordena interações

## 4.2 Classe Jardim

A classe Jardim representa o espaço onde a simulação decorre. É responsável por armazenar as posições disponíveis e as plantas existentes, bem como por disponibilizar operações que permitem a interação do jardineiro com os elementos do jardim

Jardim

├── contém Posicao

├── armazena Plantas

└── gere o espaço

## 4.3 Classe Jardineiro

A classe Jardineiro representa o agente responsável pela interação com o jardim. É através desta classe que são aplicadas ferramentas às plantas e desencadeadas as ações que influenciam a evolução do sistema ao longo da simulação.

Jardineiro

├── possui Ferramentas

├── interage com Jardim

└── executa ações

## 4.4 Classe Planta e Especializações

A classe Planta define o comportamento comum a todas as plantas do sistema, incluindo a evolução do seu estado ao longo do tempo. Através de herança, esta classe é especializada para representar diferentes tipos de plantas, cada uma com regras próprias.

Planta

├── mantém estado e vida

├── reage ao tempo

└── define comportamento base

Planta

├── Cacto

├── Roseira

├── Ervadaninha

└── PlantaExotica

## 4.5 Classe Ferramenta e Especializações

A classe Ferramenta representa uma abstração genérica para os instrumentos utilizados pelo jardineiro. As subclasses implementam comportamentos específicos, permitindo alterar o estado das plantas de diferentes formas.

Ferramenta

├── define ação base

└── atua sobre Solo (em cada posição)

└── atua sobre Planta

Ferramenta

├── Regador

├── Adubo

├── Tesoura

└── FerramentaZ

## 4.6 Características das Plantas e Ferramentas

Plantas:

| Planta | Letra no Jardim | Beleza | Água Inicial | Nutrientes Iniciais | Absorção de Água (solo) | Absorção de Nutrientes (solo) | Perda de Água por Instante | Perda de Nutrientes por Instante | Condições de Morte | Multiplicação | Efeito ao Morrer |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Cacto 🌵 | c | Neutra | 0.00 | 0.00 | 25% da água do solo | até 5 | 0 | 0 | Água no solo >100 durante 3 instantes OU nutrientes do solo = 0 durante > 3 instantes | Se água > 50 e nutrientes > 100 → cria novo cacto numa posição vizinha vazia (*1) | Deixa no solo todos os nutrientes absorvidos na sua vida |
| Roseira 🌹 | r | Bonita | 25.00 | 25.00 | 5, se existir | 8, se existir | −4 | −4 | Água ≤ 0 OU nutrientes ≤ 0 OU nutrientes ≥ 200 OU todas as posições vizinhas ocupadas | Se nutrientes >100 e existir vizinho vazio → cria nova roseira (*2) | Deixa metade da água e metade dos nutrientes que absorveu na sua vida |
| Erva Daninha 🌿 | e | Feia | 5.00 | 5.00 | 1, se existir | 1, se existir | 0 | 0 | Morre automaticamente após 60 instantes | Se nutrientes >30 e passaram 5 instantes → multiplica-se para vizinho, matando planta existente nessa nova posição (*3) | Não deixa efeito relevante no solo |
| Planta Exótica 🌺 | x | Bonita | 15.00 | 10.00 | 10, se existir | 2, se existir | −3 | -1 | Nutrientes acumulados ≤0 OU 3 instantes seguidos sem absorver água | Não se multiplica | Não deixa efeito relevante no solo |

(*1) – A água e os nutrientes presentes no cacto são divididos em iguais partes pelo cacto inicial e pelo novo

(*2) – A nova planta começa com 25 nutrientes e metade da água que a planta original tem. A roseira original fica com 100 unidades de nutrientes e com metade da água que tinha.

(*3) – A nova planta fica com 5 de água, e 5 de nutrientes, a inicial não perde nenhum destes.

Ferramentas:

| Ferramenta | Letra | Efeito Principal | Consumo / Desgaste | Duração |
|---|---|---|---|---|
| Regador 🚿 | g | +10 água no solo | Consome água do regador | Até a água acabar (200 unidades de água) |
| Adubo 🌱 | a | +10 nutrientes no solo | Consome adubo | Até o adubo acabar (100 unidades de adubo) |
| Tesoura ✂️ | t | Elimina plantas feias | Não se desgasta | Ilimitada |
| FerramentaZ ⓩ | z | Limpa as plantas mortas do jardim (letras maiúsculas) | Não se desgasta | Ilimitada |

## 4.7 Classes de Apoio

A classe Posicao é utilizada para representar a localização dos elementos no jardim. Esta abstração permite separar a lógica espacial das restantes classes, simplificando a gestão do espaço e a organização das plantas.

Posicao

├── representa coordenadas

└── identifica localização no jardim

A classe Settings agrupa os parâmetros globais da simulação, permitindo alterar comportamentos e valores de configuração sem necessidade de modificar a lógica principal do sistema. Esta abordagem contribui para maior flexibilidade e facilidade de ajuste.

Settings

├── armazena parâmetros globais

└── centraliza configurações

# 5. Funcionamento da Simulação

O funcionamento da simulação pode ser descrito de forma sequencial, desde a inicialização do sistema até à atualização contínua do estado do jardim.

1. A aplicação é iniciada através do ficheiro main.cpp, onde é criado o objeto responsável pela simulação.
2. O simulador inicializa o jardim, o jardineiro e as restantes estruturas necessárias ao funcionamento do sistema.
3. São criadas as plantas iniciais e associadas a posições específicas dentro do jardim.
4. A simulação entra num ciclo de execução, representando a passagem do tempo.
5. Em cada iteração do ciclo, o jardineiro pode executar uma ação, utilizando uma ferramenta sobre uma planta.
6. As ferramentas aplicam os seus efeitos, alterando o estado interno das plantas afetadas.
7. Após as ações do jardineiro, todas as plantas atualizam o seu estado, refletindo a evolução natural ao longo do tempo.
8. O jardim é atualizado de acordo com o novo estado das plantas, podendo ocorrer crescimento, degradação ou morte.
9. O ciclo repete-se até que a simulação termine.

# 6. Conclusão

O desenvolvimento deste trabalho permitiu aplicar de forma prática os principais conceitos da programação orientada a objetos, através da implementação de um sistema de simulação de um jardim. A modelação adotada possibilitou uma clara separação de responsabilidades entre classes, facilitando a organização e manutenção do código.

A utilização de herança permitiu representar diferentes tipos de plantas e ferramentas de forma consistente, enquanto a interação entre objetos garantiu uma simulação coerente e evolutiva. O sistema desenvolvido cumpre os requisitos definidos no enunciado, apresentando um comportamento estável e previsível ao longo da execução.

Em síntese, o trabalho contribuiu para a consolidação dos conhecimentos adquiridos na disciplina, demonstrando a eficácia da programação orientada a objetos na resolução de problemas deste tipo.